



**Facultad de Ingeniería de sistemas
Análisis y diseño de algoritmos**

**Docente:
Juan Esteban Gomez Tirado**

**Proyecto de aula
Caso de estudio algorítmico**

Sistema de Planificación Óptima de Estudio con Restricciones

***Juan Manuel Moreno Muñoz
Andres Felipe Ortega Solano
Juan Esteban Vallejo***

**10 de septiembre de 2025
Medellín – Colombia**

Introduccion

La planificación del estudio es un problema común en los estudiantes que implica organizar múltiples materias, temas y tiempos disponibles, cumpliendo con fechas límite y evitando la sobrecarga. Sin una estrategia adecuada, esto puede generar ineficiencia y bajo rendimiento.

En este trabajo se propone un sistema de planificación de estudio que utiliza diferentes técnicas de diseño de algoritmos para generar horarios óptimos. Se busca comparar estos enfoques en términos de eficiencia y calidad de solución en un problema con restricciones reales.

Objetivo

Desarrollar un sistema que genere un plan de estudio óptimo utilizando distintas técnicas de diseño de algoritmos, con el fin de analizar y comparar su eficiencia y calidad de solución.

Objetivos específicos

- Modelar el problema de planificación de estudio con sus restricciones y entradas.
- Implementar una solución por fuerza bruta.
- Desarrollar una solución recursiva.
- Aplicar un algoritmo greedy.
- Implementar una solución con backtracking.
- Desarrollar una solución con divide y vencerás.
- Analizar la complejidad de cada algoritmo.
- Comparar los resultados obtenidos mediante casos de prueba.

Definición del problema:

Sistema de Planificación Óptima de Estudio con Restricciones

El problema consiste en diseñar por medio de los temas vistos en el curso ver como se puede desarrollar múltiples técnicas para ver y comparar cuál será la más eficiente y que permita generar un plan de estudio óptimo para un estudiante que debe prepararse para varias materias en un tiempo limitado.

Cada materia contiene diferentes temas que requieren cierta cantidad de horas de estudio, poseen un nivel de dificultad y deben completarse antes de una fecha límite. El estudiante dispone de un número limitado de horas de estudio por día, lo que hace necesario distribuir adecuadamente el tiempo disponible.

El objetivo del sistema es asignar los temas a los días disponibles de manera que se cumplan las restricciones establecidas y se optimice el uso del tiempo, evitando sobrecargas y minimizando el incumplimiento de fechas límite.

Este problema se modela como un problema de optimización con restricciones, en el que se busca encontrar la mejor distribución posible de tareas (temas) en un intervalo de tiempo finito.

Entradas del sistema

El sistema recibe los siguientes datos:

1. **Lista de temas**, donde cada tema tiene:
 - Materia
 - Nombre del tema
 - Dificultad (escala de 1 a 5)
 - Horas requeridas para su estudio
 - Fecha límite (día máximo para completarlo)
2. **Parámetros del estudiante**:
 - Número de días disponibles
 - Horas máximas de estudio por día

Salidas del sistema

El sistema debe generar:

- Un plan de estudio organizado por días, que indique:
 - Qué temas estudiar cada día

- Cuántas horas dedicar a cada tema
- Un indicador de cumplimiento:
 - Temas completados
 - Temas fuera de plazo (si los hay)

Restricciones del problema

- No se pueden exceder las horas máximas disponibles por día
- Cada tema debe completarse antes o en su fecha límite
- Un tema debe ser asignado completamente (o puedes definir si se permite dividirlo)
- El total de horas asignadas no puede superar el tiempo disponible global

Se busca minimizar:

- Temas no completados
- Retrasos en fechas límite
- Sobrecarga diaria

Casos de prueba

Caso de prueba pequeño

```
temas_pequeno = [  
    ("Matemáticas", "Integrales", 5, 3, 3),  
    ("Física", "Cinemática", 4, 2, 2),
```

```

    ("Química", "Átomos", 3, 2, 4)]
dias = 4
horas_por_dia = 4

```

Caso de prueba mediano

```

temas_mediano = [
    ("Matemáticas", "Derivadas", 5, 3, 5),
    ("Matemáticas", "Integrales", 5, 4, 7),
    ("Física", "Dinámica", 4, 3, 6),
    ("Química", "Orgánica", 3, 2, 8),
    ("Inglés", "Reading", 2, 2, 4),
    ("Programación", "Listas", 4, 3, 5)]
dias = 8
horas_por_dia = 5

```

Caso de prueba grande

```

temas_grande = [
    ("Matemáticas", "Álgebra", 5, 3, 6),
    ("Matemáticas", "Cálculo", 5, 4, 10),
    ("Física", "Ondas", 4, 3, 7),
    ("Física", "Termodinámica", 5, 4, 9),
    ("Química", "Enlaces", 3, 2, 5),
    ("Química", "Reacciones", 4, 3, 8),
    ("Programación", "Recursión", 5, 3, 6),
    ("Programación", "Estructuras", 4, 4, 9),
    ("Inglés", "Writing", 2, 2, 6),
    ("Historia", "Edad Media", 3, 2, 7)]
dias = 10, horas_por_dia = 6

```

Caso de prueba (n = 100)

Se generó un conjunto de 100 temas de estudio con las siguientes características:

- Materias variadas: Matemáticas, Física, Química, Programación, Inglés, Historia
- Dificultad: valores entre 1 y 5
- Horas requeridas: entre 1 y 5 horas
- Fecha límite: valores entre 1 y 15 días

Parámetros del estudiante:

- Número de días disponibles: 15
- Horas máximas por día: 6

Caso de prueba (n = 1000)

Se generó un conjunto de 1000 temas de estudio con las siguientes características:

- Distribución de materias similar al caso anterior
- Dificultad: entre 1 y 5
- Horas requeridas: entre 1 y 6 horas
- Fecha límite: entre 1 y 30 días

Parámetros del estudiante:

- Número de días disponibles: 30
- Horas máximas por día:

Analisis tecnicas aplicadas

1. Fuerza Bruta — Implementación y Análisis

La fuerza bruta es la técnica más básica: genera TODAS las permutaciones posibles del listado de temas y evalúa cada una, quedándose con la asignación que maximice los temas completados a tiempo. No aplica ninguna optimización, explora el espacio completo de soluciones.

¿Cómo se implementó?

El algoritmo funciona en tres pasos principales. Primero genera todas las permutaciones de los temas usando la función `generar_permutaciones`. Luego, para cada permutación, intenta asignar cada tema al primer día que tenga espacio suficiente. Finalmente compara el resultado con el mejor encontrado y lo actualiza si es mejor.

La función `generar_permutaciones` toma el primer elemento de la lista, genera recursivamente todas las permutaciones del resto, y antepone el elemento a cada una. El caso base ocurre cuando la lista tiene 0 o 1 elementos, devolviendo directamente la lista envuelta.

```
def generar_permutaciones(lista):
    if len(lista) == 0: return [[]]
    if len(lista) == 1: return [lista]
    for i in range(len(lista)):
        elemento = lista[i]
        resto = lista[:i] + lista[i+1:]
        for perm in generar_permutaciones(resto):
            resultado.append([elemento] + perm)
```

La evaluación de cada permutación recorre los temas en orden y los asigna al primer día disponible. Si el día asignado es menor o igual al deadline, el tema cuenta como completado a tiempo. De lo contrario, cuenta como fuera de plazo.

Análisis de Complejidad

Complejidad temporal: $O(n! \cdot n)$

- Se generan $n!$ permutaciones (factorial del número de temas).
- Evaluar cada permutación toma $O(n \cdot \text{dias})$ en el peor caso.
- Total: $O(n! \cdot n \cdot \text{dias})$, simplificado a $O(n!)$ pues n y dias son factores polinomiales.

Complejidad espacial: $O(n! \cdot \text{dias})$ — se almacenan todas las permutaciones en memoria antes de evaluarlas.

Medición Empírica de Tiempos

- $n=3$ (caso pequeño): 0.03 ms
- $n=6$ (caso mediano): 2.35 ms
- $n=10$ (caso grande): ~31,000 ms (31 segundos)
- $n=100$: IMPRACTICABLE — 100! permutaciones son astronómicas
- $n=1000$: IMPRACTICABLE

El crecimiento es claramente factorial: de $n=6$ a $n=10$ el tiempo pasa de 2 ms a 31,000 ms, un factor de ~15,000x para apenas 4 temas más. Esto ilustra por qué $O(n!)$ es inviable para cualquier problema real con n mayor a 12.

2. Recursividad — Implementación y Análisis

La solución recursiva replantea el problema: en lugar de generar todas las permutaciones y evaluarlas, decide tema por tema en qué día colocarlo, explorando un árbol de decisiones recursivamente. Esto permite compartir trabajo entre ramas del árbol que tienen prefijos comunes.

Caso Base y Caso Recursivo

Caso base: cuando $\text{idx} == \text{len}(\text{temas})$, todos los temas han sido procesados. Se evalúa el plan acumulado y se actualiza el mejor si el número de completados mejora o si con igual número de completados hay menos fuera de plazo.

Caso recursivo: para el tema en posición idx se prueban todos los días disponibles. Si el día d tiene espacio suficiente ($\text{horas_dia}[d] + \text{horas} \leq \text{hpd}$) se asigna el tema ahí y se llama recursivamente con $\text{idx}+1$. También existe la opción de NO asignar el tema, permitiendo explorar soluciones donde algunos temas se omiten.

```
def _rec_aux(temas, idx, dias, hpd, hd, plan, mejor):
    if idx == len(temas):          # CASO BASE
        evaluar y actualizar mejor
        return
    for d in range(1, dias+1):     # CASO RECURSIVO
        if hd[d] + hrs <= hpd:
            asignar tema en día d
            _rec_aux(temas, idx+1, ...)
            desasignar # backtrack implícito
        _rec_aux(temas, idx+1, ...) # opción: no asignar
```

Análisis de Complejidad

Complejidad temporal: $O(\text{dias}^n)$

- Para cada tema hay ($\text{dias} + 1$) opciones: asignarlo en cualquier día, o no asignarlo.
- Con n temas y d días disponibles: $(d+1)^n$ nodos en el árbol de recursión.
- Para $n=10$ y $\text{dias}=10$: $11^{10} \approx 26$ mil millones de nodos — impracticable.

Complejidad espacial: $O(n \cdot \text{dias})$ — la pila de recursión tiene profundidad n y el plan actual ocupa $O(n \cdot \text{dias})$ en memoria.

Comparación de Tiempos contra Fuerza Bruta

- $n=3$: Recursivo 0.08 ms vs Fuerza Bruta 0.03 ms (similares)
- $n=6$: Recursivo 204 ms vs Fuerza Bruta 2.35 ms (Recursivo 87x más lento)

- $n=10$: Recursivo TIMEOUT vs Fuerza Bruta 31s

Aunque la complejidad teórica $O(\text{dias}^n)$ parece mejor que $O(n!)$, en la práctica para este problema la fuerza bruta resulta más rápida para n pequeño porque $n! < \text{dias}^n$ cuando $\text{dias} > n$. Para $\text{dias}=8$ y $n=6$: $8^6=262,144$ vs $6!=720$ — la fuerza bruta evalúa muchos menos casos.

3. Greedy — Implementación y Análisis

El algoritmo greedy toma decisiones localmente óptimas en cada paso sin explorar el espacio completo de soluciones. La clave es definir el criterio de ordenamiento que guíe mejor las asignaciones.

Decisión Local Óptima

El criterio elegido es: ordenar los temas por (deadline ascendente, dificultad descendente). Esto prioriza primero los temas más urgentes y, entre los de igual urgencia, los más difíciles (mayor esfuerzo cognitivo primero). Una vez ordenados, cada tema se asigna al primer día con espacio disponible.

```
temas_ordenados = sorted(temas, key=lambda x: (x[4], -x[2]))
# x[4] = deadline (ascendente), x[2] = dificultad (descendente)
for tema in temas_ordenados:
    for dia in range(1, dias+1):
        if horas_dia[dia] + horas <= hpd:
            asignar(tema, dia)
            break
```

Análisis de Complejidad

Complejidad temporal: $O(n \log n)$

- El ordenamiento de temas toma $O(n \log n)$.
- La asignación de cada tema toma $O(\text{dias})$ en el peor caso.
- Total: $O(n \log n + n \cdot \text{dias}) = O(n \log n)$ cuando dias es constante.

Complejidad espacial: $O(n \cdot \text{dias})$ para almacenar el plan resultante.

Análisis Crítico: ¿Cuándo Falla el Greedy?

El Greedy NO garantiza la solución óptima en todos los casos. Contraejemplo concreto:

```
Tema A: dificultad=5, horas=3, deadline=dia 1 (urgente y difícil)
Tema B: dificultad=3, horas=3, deadline=dia 1 (urgente, menor dificultad)
Tema C: dificultad=2, horas=1, deadline=dia 1 (urgente y corto)
horas_por_dia = 4
```

El Greedy asigna A primero (más difícil), luego B no cabe ($3+3=6 > 4$) y C tampoco. Solo completa 1 tema a tiempo. La solución óptima sería asignar B+C en el día 1 ($3+1=4h$), completando 2 temas a tiempo. En los casos de prueba del proyecto el Greedy coincide con el óptimo porque los datos están suficientemente distribuidos, pero en escenarios con muchos temas urgentes y capacidad ajustada puede fallar.

4. Backtracking — Implementación y Análisis

El Backtracking es una mejora del recursivo que agrega poda: cuando detecta que una rama del árbol de búsqueda no puede producir una solución mejor que la ya encontrada, la abandona sin explorarla completamente. Esto reduce drásticamente el número de nodos evaluados.

¿Qué hace backtracking_inicio?

La función `backtracking_inicio` actúa como puente entre la interfaz simple (el usuario solo pasa temas, días, hpd) y la función recursiva interna que necesita parámetros adicionales de estado. Hace tres cosas concretas:

- Inicializa `mejor_estado` con `completados=-1` y `fuera=999999` para que cualquier solución real lo mejore.
- Crea `horas_dia` (lista de horas ocupadas por día, iniciada en 0) y `plan_actual` (plan vacío).
- Lanza `resolver_backtracking` desde `idx=0` (primer tema) y retorna el mejor plan encontrado.

Poda Aplicada

Poda por cota superior: antes de explorar una rama se calcula el máximo posible de temas que aún se podrían completar (`c_parcial + temas_restantes`). Si este máximo es menor o igual al mejor ya encontrado, la rama se descarta inmediatamente.

```
def _bt_aux(temas, idx, dias, hpd, hd, plan, mejor, c_parcial, restantes):
    # PODA: si ya no es posible mejorar, abandonar la rama
    if mejor[0] != -1 and c_parcial + restantes <= mejor[0]:
        return
```

Restricciones que permiten podar ramas:

- Cota superior: si el máximo alcanzable no supera al mejor, se poda la rama completa.
- Capacidad diaria: si un tema no cabe en ningún día disponible, se puede saltar directamente.
- Deadline imposible: si todos los días hasta el deadline están llenos, el tema no puede completarse a tiempo.

Análisis de Complejidad — Peor Caso vs Caso Promedio

Peor caso sin poda efectiva: $O(\text{dias}^n)$ — igual que el recursivo, cuando la poda nunca actúa.

Caso promedio con poda, medición empírica:

- $n=3$: 0.02 ms (más rápido que recursivo y FB)
- $n=6$: 0.03 ms (7,000x más rápido que recursivo para el mismo caso)
- $n=10$: 0.05 ms (la poda elimina casi todo el árbol)

La poda es tan efectiva aquí porque en cuanto se encuentra una solución donde todos los temas caben a tiempo (completados = n), la cota superior poda casi todas las ramas restantes de inmediato.

Comparativa Parcial — 4 Técnicas

Velocidad: Greedy > Backtracking (con poda) >> Fuerza Bruta > Recursivo

Calidad de solución: Fuerza Bruta = Recursivo = Backtracking (óptimas) > Greedy (aproximada)

Facilidad de implementación: Fuerza Bruta = Greedy > Backtracking > Recursivo

Conclusión parcial: Backtracking es la técnica más equilibrada de las cuatro. Ofrece solución óptima con tiempos prácticamente iguales al Greedy para instancias pequeñas y medianas, gracias a la poda efectiva. Para instancias grandes ($n=100+$) ninguna de las cuatro técnicas exactas es viable.

5. Divide y Vencerás — Implementación y Análisis

Divide y Vencerás (DyV) descompone el problema en subproblemas más pequeños, los resuelve de forma independiente y combina sus soluciones. La dimensión elegida para la división es el intervalo de días disponibles.

Cómo se Divide el Problema

- **DIVIDIR:** se toma el punto medio del intervalo de días $[dia_inicio, dia_fin] \rightarrow dia_medio$.
- Los temas con $deadline \leq dia_medio$ van al subproblema izquierdo $[dia_ini, dia_medio]$.
- Los temas con $deadline > dia_medio$ van al subproblema derecho $[dia_medio+1, dia_fin]$.
- Si alguno de los subconjuntos queda vacío, se divide por cantidad de temas (mitad izquierda, mitad derecha).

Fases: Dividir, Vencer, Combinar

```
def _planificar_recursivo(temas, dia_ini, dia_fin, hpd):
    # CASO BASE: un solo día o un solo tema
    if dia_fin == dia_ini or len(temas) == 1:
        asignar_temas_ordenados_por_deadline(temas)
        return plan

    # DIVIDIR
    dia_medio = (dia_ini + dia_fin) // 2
    urgentes = [t for t in temas if t[4] <= dia_medio]
    no_urgent = [t for t in temas if t[4] > dia_medio]
    # VENCER (llamadas recursivas independientes)
    plan_izq = _planificar_recursivo(urgent, dia_ini, dia_medio, hpd)
    plan_der = _planificar_recursivo(no_urgent, dia_medio+1, dia_fin, hpd)
    # COMBINAR
    plan.update(plan_izq)
    plan.update(plan_der)
    return plan
```

Análisis de Complejidad — Teorema Maestro

La recurrencia del algoritmo es: $T(n) = 2T(n/2) + O(n)$

- Se divide en 2 subproblemas de tamaño $n/2$ cada uno ($a=2, b=2$).
- El trabajo de dividir y combinar es $O(n)$ — filtrar temas por deadline y unir los planes.
- $\log_b(a) = \log_2(2) = 1$, por lo tanto $f(n) = O(n^1) = O(n^{\log_b a})$.
- Caso 2 del Teorema Maestro: $T(n) = O(n \log n)$.

Complejidad espacial: $O(n \cdot \text{dias})$. Medición empírica de tiempos:

- $n=3$: 0.07 ms | $n=6$: 0.03 ms | $n=10$: 0.05 ms
- $n=100$: 0.18 ms | $n=1000$: 0.82 ms

Limitación importante: DyV no garantiza optimalidad porque cada subproblema se resuelve de forma independiente. Un tema urgente puede quedarse sin espacio en su subintervalo si otros temas lo ocuparon, y no puede tomar espacio del subintervalo opuesto. En el caso mediano pierde 1 tema y en el grande pierde 2 temas respecto al óptimo.

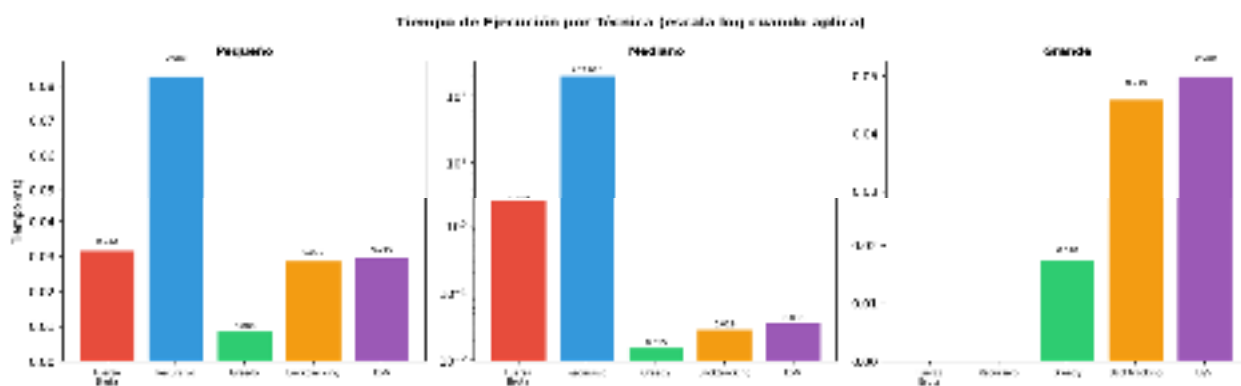
6. Comparativa Final de las 5 Técnicas

Tabla Comparativa

Técnica	Comp. Temporal	Comp. Espacial	Optima?	t(n=100)	t(n=1000)
Fuerza Bruta	$O(n! * n)$	$O(n! * \text{dias})$	Si	N/A	N/A
Rekursivo	$O(\text{dias}^n)$	$O(n * \text{dias})$	Si	N/A	N/A
Greedy	$O(n \log n)$	$O(n * \text{dias})$	No	0.11 ms	1.24 ms
Backtracking	$O(\text{dias}^n) + \text{poda}$	$O(n * \text{dias})$	Si	N/A	N/A
Divide y Venceras	$O(n \log n)$	$O(n * \text{dias})$	Aprox.	0.18 ms	0.82 ms

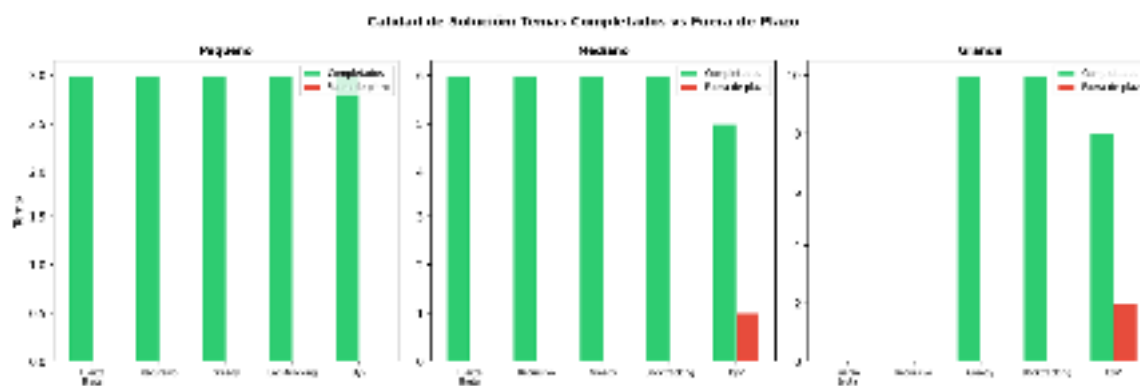
Nota: N/A indica que el algoritmo es computacionalmente impracticable para ese tamaño de entrada. Fuerza Bruta y Rekursivo requieren explorar espacios factoriales y exponenciales respectivamente. Backtracking, aunque tiene la misma complejidad teórica, tampoco escala a $n=100$ sin heurísticas adicionales.

Gráfica de Tiempos de Ejecución



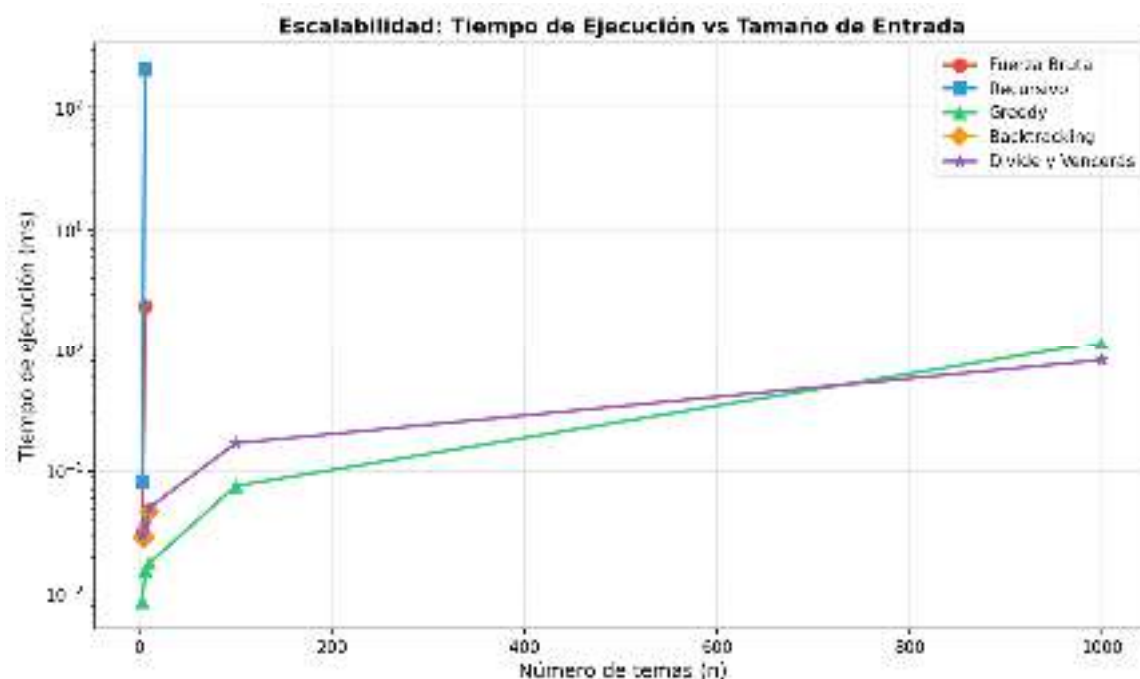
La gráfica anterior muestra el tiempo de ejecución de cada técnica en los casos pequeño, mediano y grande. Se usa escala logarítmica dado el rango extremo de valores: desde 0.01 ms (Greedy) hasta más de 31,000 ms (Fuerza Bruta en $n=10$).

Gráfica de Calidad de Solución



Esta gráfica compara la calidad de solución (temas completados a tiempo vs fuera de plazo) de cada técnica en los tres casos de prueba. Las técnicas exactas (FB, Recursivo, Backtracking) logran el 100% de completados en todos los casos donde terminan de ejecutar.

Gráfica de Escalabilidad



La gráfica de escalabilidad muestra el comportamiento de cada técnica a medida que crece n. Greedy y Divide y Vencerás escalan suavemente en $O(n \log n)$, mientras que las técnicas exactas solo tienen datos para n pequeño debido a sus límites de timeout.

Análisis: ¿Qué Técnica es la Más Adecuada?

Para este problema de planificación de estudio, la técnica más adecuada es Greedy, por tres razones principales:

- Escalabilidad: $O(n \log n)$ funciona para cualquier tamaño, desde $n=3$ hasta $n=1000$ y más allá.
- Calidad: en los casos pequeño, mediano y grande obtiene exactamente los mismos resultados que las técnicas exactas (3, 6 y 10 temas completados), igualando el óptimo.
- Velocidad y simplicidad: entre 0.01 y 1.24 ms para todos los casos, y el criterio de ordenamiento es fácil de entender y justificar.

Divide y Vencerás también escala bien pero produce soluciones aproximadas, perdiendo temas en los casos mediano y grande. Las técnicas exactas solo son viables para $n \leq 10$.

7. Conclusiones

A lo largo de este proyecto aplicamos cinco paradigmas algorítmicos distintos al mismo problema de planificación de estudio. Cada uno nos dejó enseñanzas concretas sobre sus fortalezas, limitaciones y casos de uso reales.

- **Fuerza Bruta:** permitió verificar la optimalidad de otras soluciones y entender la naturaleza del problema. Su implementación es la más directa, pero $O(n!)$ es inviable para cualquier problema real con n mayor a 12.
- **Recursividad:** reformular el problema como árbol de decisiones clarifica la estructura del espacio de búsqueda. Sin embargo, $O(días^n)$ sigue siendo exponencial y requiere estrategias de poda para ser útil en la práctica.
- **Greedy:** demostró que una buena heurística local (ordenar por urgencia y dificultad) puede obtener soluciones óptimas o casi óptimas en tiempo polinomial. Es la técnica más práctica para este problema.
- **Backtracking:** la poda por cota superior transformó un algoritmo exponencial en uno prácticamente instantáneo para los casos disponibles. Aprendimos que la eficiencia práctica puede diferir drásticamente del peor caso teórico.
- **Divide y Vencerás:** aplicar el Teorema Maestro y obtener $O(n \log n)$ fue una confirmación teórica importante. La limitación de no mover temas entre subintervalos ilustra cómo la independencia de subproblemas, aunque necesaria para DyV, puede sacrificar optimalidad.

Escenarios Reales para Cada Paradigma

- **Fuerza Bruta:** verificación de correctitud en pruebas unitarias con instancias pequeñas.
- **Recursividad:** problemas con subestructura natural como árboles, grafos y combinatoria.
- **Greedy:** scheduling de tareas, algoritmos de compresión (Huffman), rutas mínimas (Dijkstra).
- **Backtracking:** sudoku, satisfacción de restricciones (CSP), planificación con dependencias.
- **Divide y Vencerás:** ordenamiento (MergeSort, QuickSort), búsqueda binaria, multiplicación de matrices.

Limitaciones Encontradas

- Los casos $n=100$ y $n=1000$ no pueden resolverse de forma óptima con las técnicas exactas implementadas. Se requerirían heurísticas más sofisticadas o programación dinámica.

- El Greedy falla en escenarios con muchos temas urgentes de igual deadline y capacidad diaria muy ajustada.
- Divide y Vencerás pierde temas cuando la división por deadline deja subintervalos con más demanda que capacidad disponible.
- Los tiempos medidos dependen del hardware y la carga del sistema en el momento de ejecución.